Computer Hardware

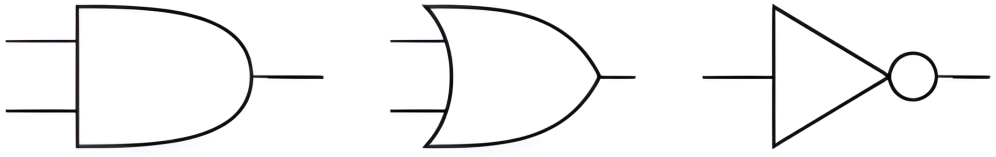
Introduction to computer programming

Management and Artificial Intelligence

Boolean operators as logic gates

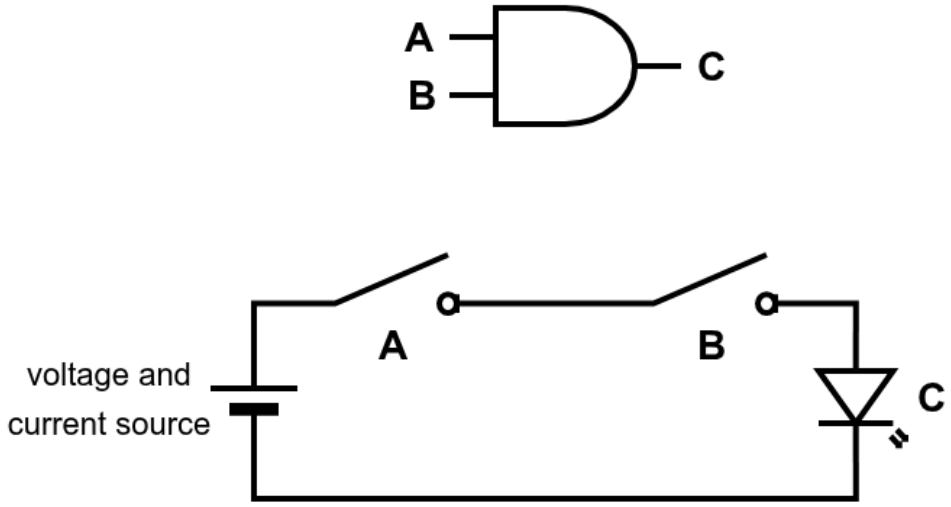
The boolean operators AND, OR, and NOT, introduced in the previous lecture, can be implemented as logic gates (circuits).

Within a circuit, they are represented by these symbols:



AND gate

The AND logic gate can be implemented with two switches in series:



The two inputs, A and B, control the two switches:

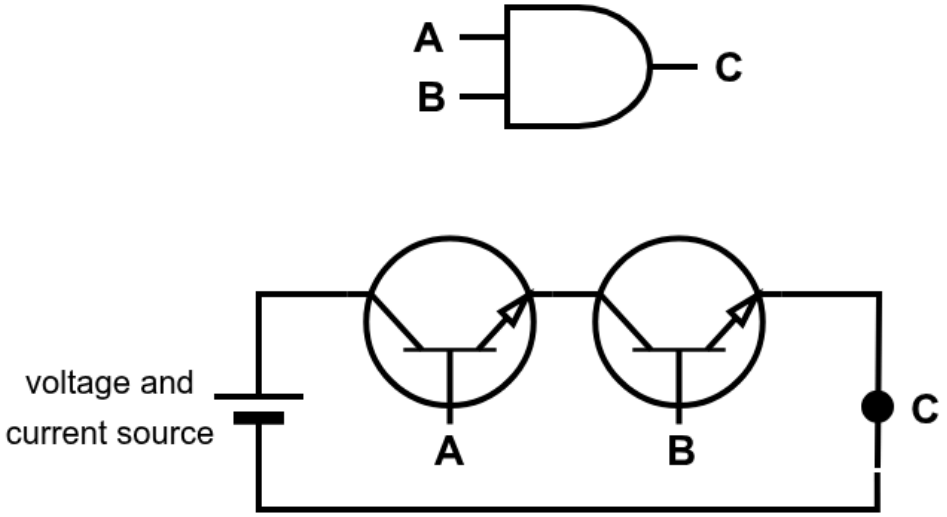
- if their boolean value is True, the switch is closed and current flows
- if their boolean value is False, the switch is open and no current flows

Only when both switches are closed (A=True and B=True), the current flows to C (represented with a LED, which would emit light to represent a True value).

NOTE: A resistor should be added in series with the LED to limit the current and prevent damage.

AND gate: from switches to transistors

In electronics, when realizing an AND gate, switches are replaced with transistors:



We have replaced the two switches with two BJT NPN transistors:

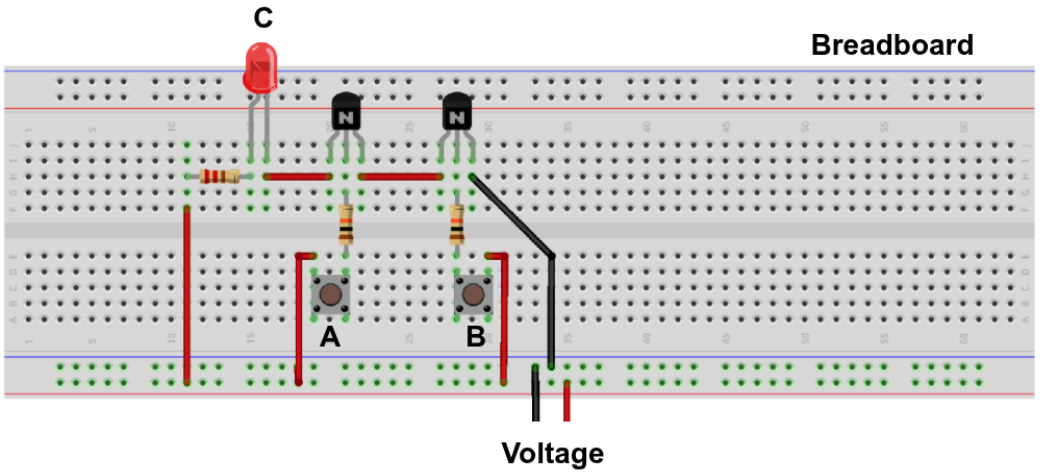
- if their base signal (A or B) is high, the transistor is on and current flows
- if their base signal (A or B) is low, the transistor is off and no current flows

Only when both transistors are on, the output signal C is high.

NOTE: There exist other types of transistors, which can be used to implement an AND gate.

AND gate: real-world (inefficient) implementation

We can buy transistors, resistors, and LED to make our own AND gate with minimal cost: "electronics starter kit" (often sold with Arduino boards) cost less than 15-20€ and allows to build simple circuits.



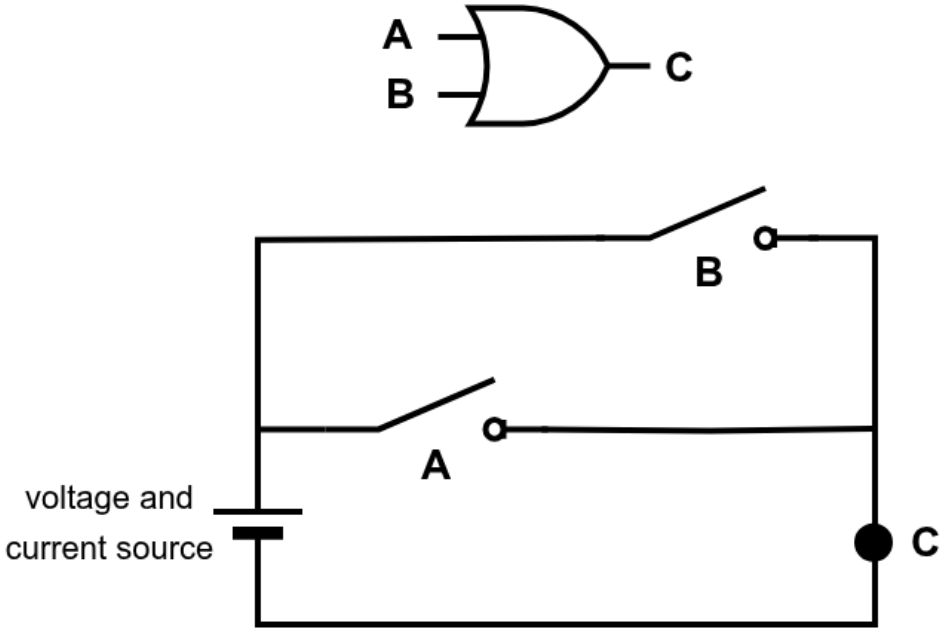
- the breadboard is a device that allows you to connect the components without soldering them
- we added some resistors to limit the current flowing through the LED
- we can manually control A and B by pushing the buttons
- the LED will light up if and only if both A and B are pressed

In the real world:

- A and B would be generated by other circuits, without the need of manually controlling them!
- C would be used to likely control other circuit or generate a bit value

OR gate

The OR logic gate can be implemented with two switches in parallel:



The two inputs, A and B, control the two switches:

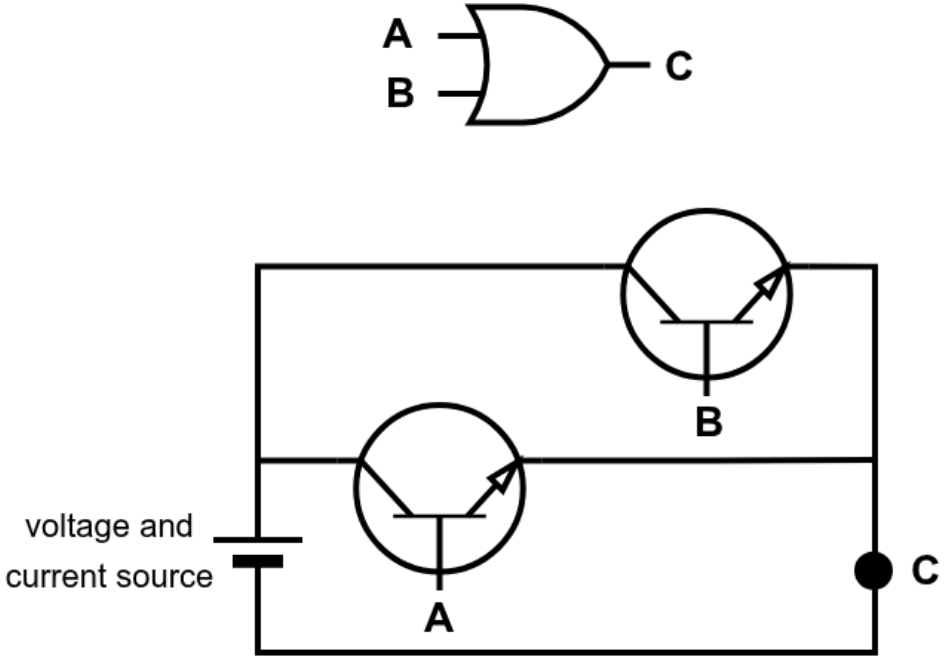
- if their boolean value is True, the switch is closed and current flows
- if their boolean value is False, the switch is open and no current flows

When at least one switch is closed (A=True or B=True), the current flows to C (represented with a LED, which would emit light to represent a True value).

NOTE: A resistor should be added in series with the LED to limit the current and prevent damage.

OR gate: from switches to transistors

In electronics, when realizing an OR gate, switches are replaced with transistors:



We have replaced the two switches with two BJT NPN transistors:

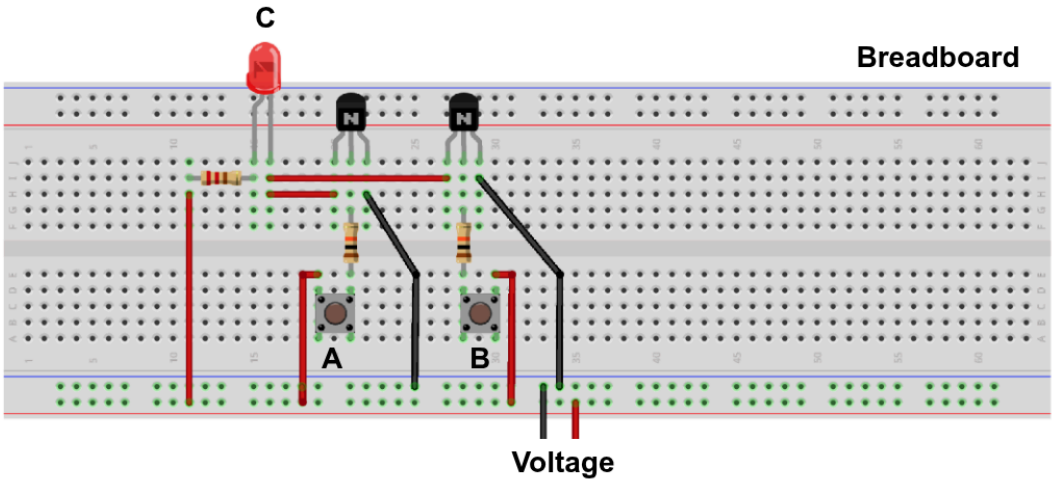
- if their base signal (A or B) is high, the transistor is on and current flows
- if their base signal (A or B) is low, the transistor is off and no current flows

When at least one transistors is on, the output signal C is high.

NOTE: There exist other types of transistors, which can be used to implement an AND gate.

OR gate: real-world (inefficient) implementation

As for the AND gate, we can implement the OR gate with a few cheap transistors, resistors, and LED.



- the breadboard is a device that allows you to connect the components without soldering them
- we added some resistors to limit the current flowing through the LED
- we can manually control A and B by pushing the buttons
- the LED will light up if at least one of A or B is pressed

In the real world:

- A and B would be generated by other circuits, without the need of manually controlling them!
- C would be used to likely control other circuit or generate a bit value

What can we do with AND, OR, NOT gates?

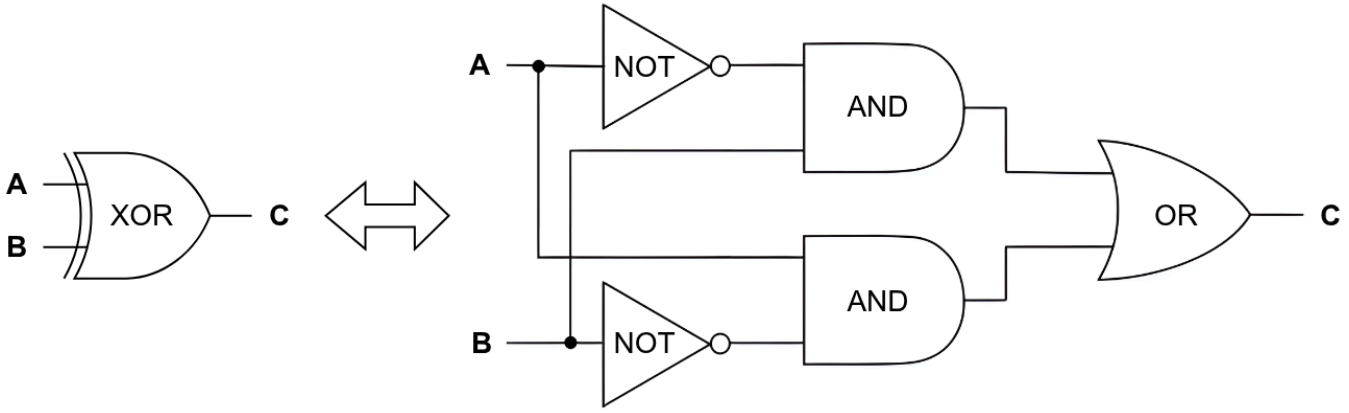
We can combine AND, OR, and NOT gates to implement a logic circuit that implements arbitrary boolean functions.

For instance, we see how to implement two common components:

1. **Half Adder**: a circuit that implements the addition of two binary numbers.
2. **Full Adder**: a circuit that implements the addition of two binary numbers with a carry input.

XOR gate

To easily implement an adder, we need to implement an XOR gate:



As seen in the previous lecture, a XOR can be implemented by combining AND, OR, and NOT gates:

XOR(A, B) = OR(AND(A, NOT(B)), AND(NOT(A), B))

Half Adder: boolean function

We want to design a circut that:

- camputes the sum of two binary numbers A and B
- the output is:
    - SUM(A, B): the sum of A and B
    - CARRY(A, B): the carry of SUM(A, B)

A	B	SUM(A, B)	CARRY(A, B)
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

NOTE:

- when both A and B are 1, the output bit is not enough to represent the sum which would be 10_2, hence, the output is split into two bits: the sum and the carry.

Half Adder: circuit implementation

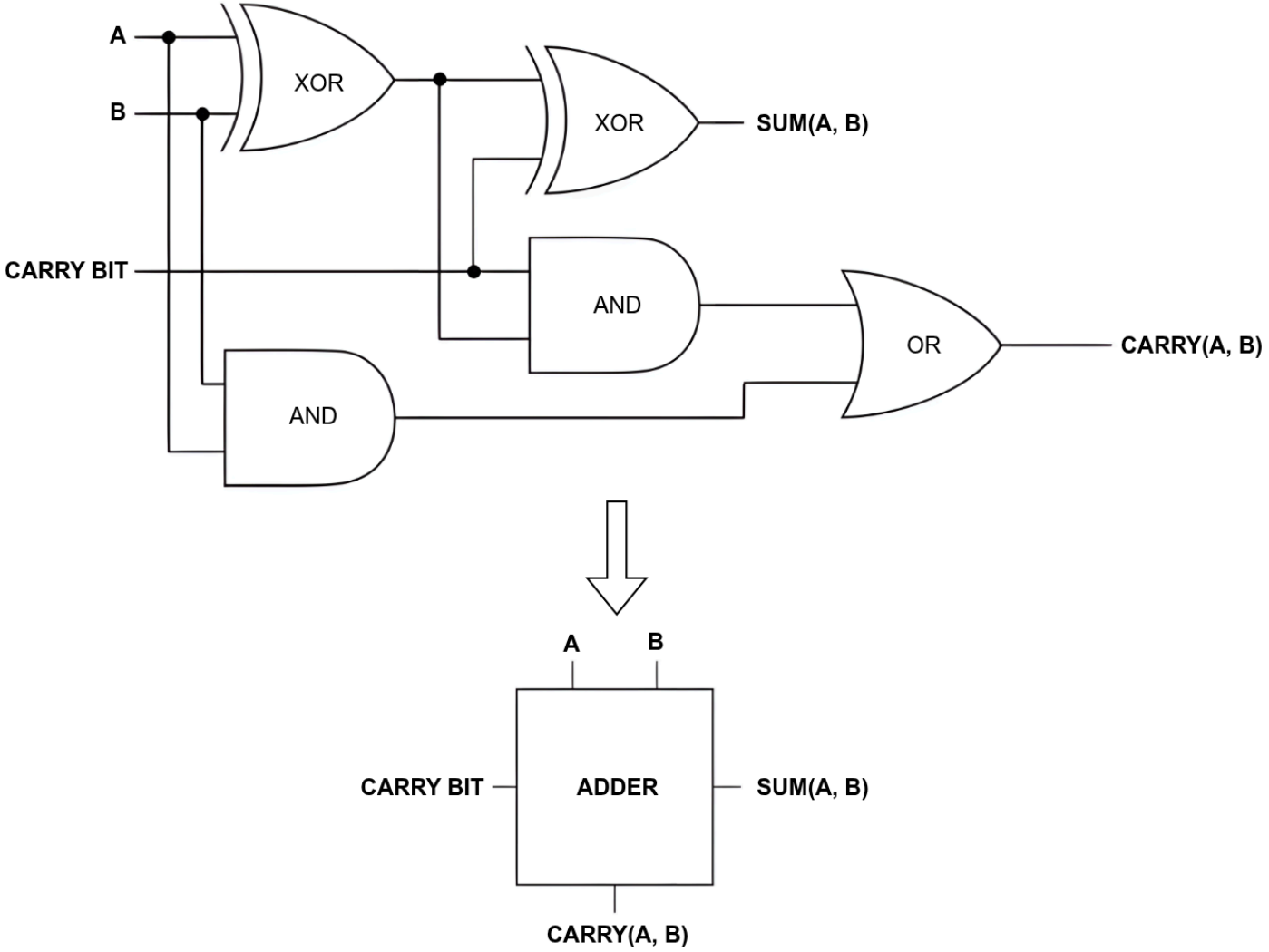
Full Adder: boolean function

We want to design a circut that:

- camputes the sum of two binary numbers A and B
- the output is:
    - SUM(A, B): the sum of A and B $\Rightarrow 0_2 + 0_2 = 0_2, 0_2 + 1_2 = 1_2, 1_2 + 1_2 = 0_2$
    - CARRY(A, B): the carry of SUM(A, B)
- if are doing a sequence of additions, we have to also consider a carry from the previous addition
    - SUM(SUM(A, B), CARRY)

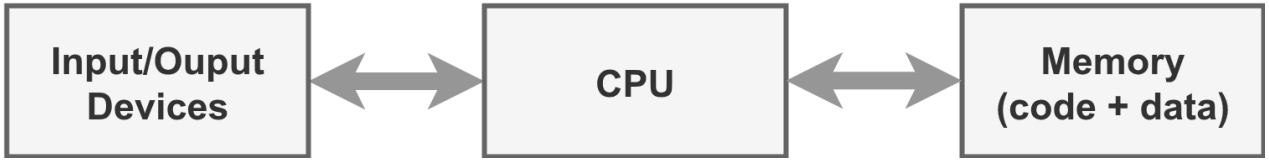
A	B	CARRY	CARRY(A, B)	SUM(A, B)
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

Full Adder: circuit implementation



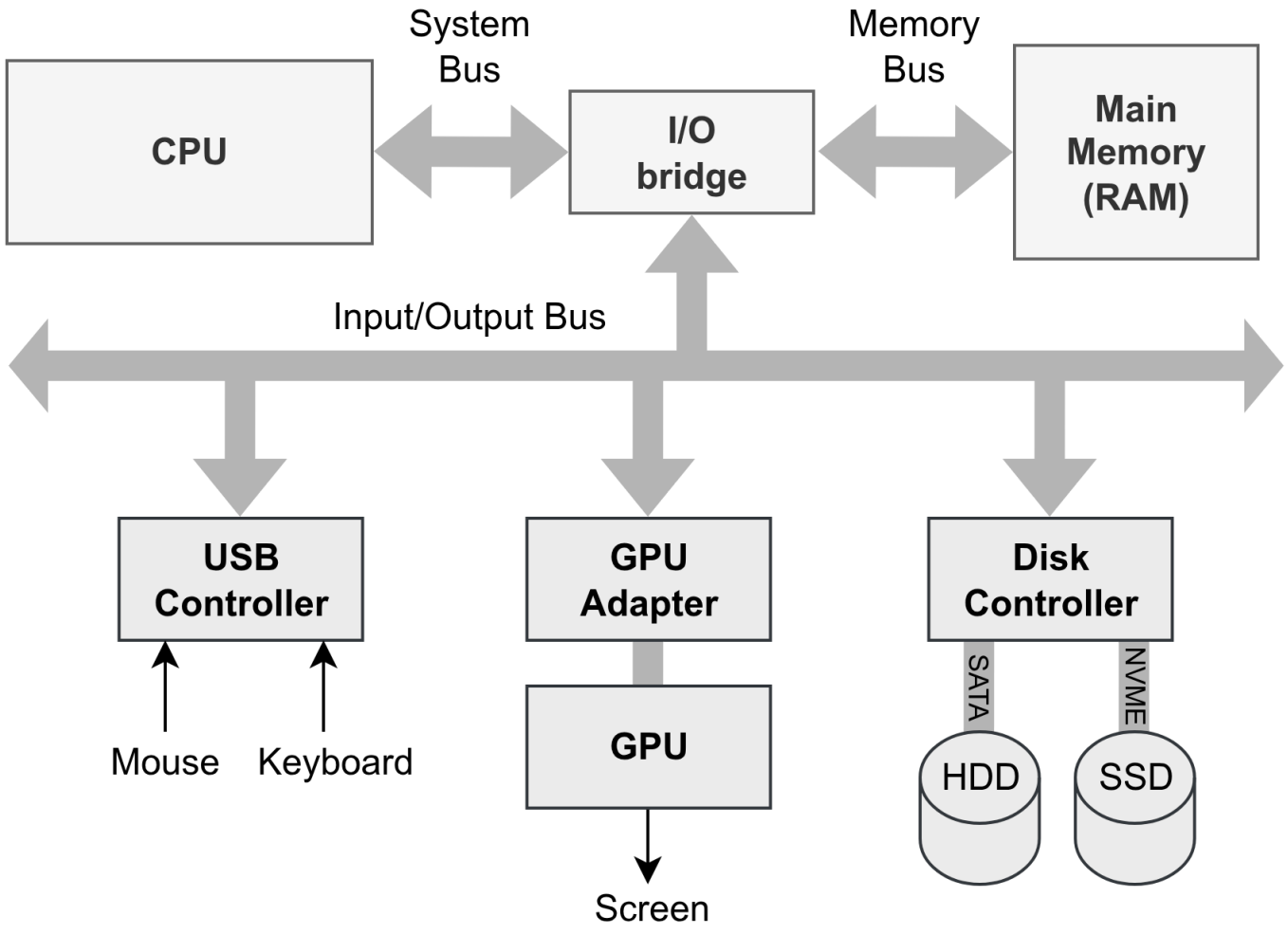
How does the hardware of a computer work?

An (still accurate) high level overview of a computer:



von Neumann architecture (1945)

A more detailed overview of a computer



We all knew the hardest part...